LexManage

B.Tech Project Defense — Preparation Guide

Speaking Script • Technology Rationale • Jury Questions & Answers

A multi-tenant SaaS legal-management platform with AI-powered legal research, built with React and NestJS and deployed on a fully free cloud stack.

Prepared for upload into NotebookLM • Author: Emmanuel Nyouma

How to use this document

1) Read Part 1 to learn what to *say* on each slide. 2) Use Part 2 to defend every technology choice. 3) Rehearse Part 3 — the jury questions — until the answers feel natural. 4) Part 4 lists smart things to add. Upload the whole file to NotebookLM and ask it to quiz you.

Contents

Part 1 — What to Say on Each Slide

Below is a simple, word-by-word style script for every section of your presentation. You do not need to memorise it. Read it a few times, understand the meaning, and say it in your own words. Each block matches one slide of your PowerPoint.

Slide 1 — Title

"Good morning. My project is called **LexManage**. It is a software platform that helps a law firm run its whole daily work in one secure place — managing cases, storing legal documents, keeping client records, sending reminders, and even answering legal questions using artificial intelligence. It is built as a **multi-tenant SaaS**, which means many different law firms can use the same application, but each firm's data stays completely separate and private. My name is Emmanuel Nyouma, and I will now take you through the problem, the solution, the technology, and a live demonstration."

Slide 2 — Agenda

"Here is the plan for my presentation. First I will introduce LexManage. Then I will explain the real problems that law firms face today. After that I will show the features and explain exactly which feature solves which problem. Next I will describe the methodology and all the tools I used — from writing the code to putting it online. Then we will look at the results with a live demo, and finally I will conclude."

Slide 3 — Introduction: What is LexManage

"Let me explain what LexManage really is. It is an **enterprise-grade platform** — meaning it is built to professional standards — that turns the paperwork of a law firm into organised digital information. The most important idea here is the word **tenant**. A tenant is one law firm. When a firm signs up, it gets its own private space inside the system: its own lawyers, its own clients, its own cases and documents. One firm can never see another firm's data. This separation is enforced deep inside the software, on every single request."

"On the right of this slide you can see the size of the project: it has **17 backend modules**, **13 database models**, **5 levels of user roles**, and one AI assistant that answers using the firm's own documents. The platform also works in **both French and English**, has a dark mode, and is fully usable on a mobile phone."

Slide 4 — Problem Statement

"Why did I build this? Because most small and medium law firms — especially here in Africa — still work with paper files, documents scattered across different computers and USB drives, spreadsheets, and WhatsApp or email. This creates real, daily problems."

"**One**: information about a case is fragmented — the facts, the parties, the deadlines and the history are spread everywhere, so there is no single source of truth. **Two**: documents are disorganised and insecure — they are hard to find and shared with no access control. **Three**: hearings and deadlines are tracked by hand, so critical dates are missed, and in law a missed deadline can lose a case. **Four**: legal research is slow — finding one clause across hundreds of

pages takes hours. **Five**: when firms share a digital tool, there is a real danger that one firm could see another firm's confidential data. And **six**: without roles and permissions, every user can see and change everything, which is unacceptable for confidential legal data."

Slide 5 — Features Overview

"To answer those problems, LexManage has eight core features. **Case Management** tracks the full life of a case. The **Document Management System** lets you upload, categorise and secure legal files. The **Client Directory** is a mini-CRM for all clients. **LexAssist AI** answers legal questions from the firm's own documents. The **Notification Center** sends instant, scheduled and template-based alerts. **Hybrid Smart Search** finds anything quickly with live suggestions. The **Calendar** keeps hearings and deadlines in one view. And **Role-Based Access Control with multi-tenancy** keeps each firm's data private and gives each user only the rights they need."

Slide 6 — Which Feature Solves Which Problem

"This slide is the heart of my project, because every feature was built to answer a specific problem. Fragmented information is solved by Case Management and the Calendar — one organised file per case. Disorganised documents are solved by the Document Management System, with categories, per-role access, and secure download links that expire after 15 minutes. Missed deadlines are solved by the deadline tracker and scheduled notifications that fire automatically at the right time. Slow research is solved by LexAssist AI. Privacy between firms is solved by multi-tenancy. Poor coordination is solved by real-time notifications. And the lack of access control is solved by Role-Based Access Control. Nothing in this system is decoration — each feature has a job."

Slide 7 — Methodology (How It Was Built)

"I built LexManage in a structured, step-by-step way. **Step one**: I modelled the legal world as a database — 13 connected tables for firms, users, cases, clients, documents, deadlines and notifications. **Step two**: I built the backend API first, one module per topic, and I validated every input. **Step three**: I built the user interface as reusable components. **Step four**: I added security everywhere — login, roles, data isolation, rate limiting. **Step five**: I tuned performance with caching and smart pagination. **Step six**: I deployed everything to the cloud with automatic updates on every code change. The key idea is that **security and structure came first, not last**."

Slide 8 — End-to-End Architecture

"This is how the whole system fits together. On the left is the **client** — the part that runs in the user's browser, built with React and served from a global content network so it loads fast. In the middle is the **API** — the brain of the system, built with NestJS, which checks who you are, what firm you belong to, and what you are allowed to do. On the right are the **data and services** — the PostgreSQL database, the Redis cache and job queue, the file storage, and the AI workflow. Everything talks over HTTPS, every request carries a secure token, and every request is filtered so a firm only ever sees its own data."

Slide 9 — Frontend & Backend Technology

"On the frontend I used **React 19 with Vite** for a fast, modern interface, **Tailwind CSS** for the design, **Zustand** to remember the logged-in user, and **React Query** to talk to the server smartly with caching and automatic retries. Forms are validated with **Zod** — and importantly, the same

validation rules run on the server too, so there is one single source of truth for what valid data looks like."

"On the backend I used **NestJS**, a professional framework that organises code into modules. The database is **PostgreSQL**, accessed through **Prisma**, which makes the queries safe from SQL injection automatically. Logins use **JWT tokens**, passwords are protected with **bcrypt**, background jobs run on **BullMQ with Redis**, real-time updates use **Socket.io**, and the API is protected by security headers, rate limiting and strict input checks."

Slide 10 — Data, Storage & Security

"Let me go deeper on data and security, because for a legal app this is everything. All structured data lives in PostgreSQL. All documents are stored in S3-compatible storage, and every file is saved under a folder named after the firm's ID, so one firm's files can never mix with another's. Documents can only be downloaded through **signed links that expire after 15 minutes**."

"For security: every login token is cryptographically signed and verified with a fixed algorithm, so it cannot be faked. Passwords are hashed with bcrypt, never stored as plain text. There are five user roles, and admin actions are blocked for normal users. The server uses security headers, limits how many requests a user can make to stop abuse, forces HTTPS, and only accepts requests from our official website address. Sensitive actions are written to an audit log."

Slide 11 — LexAssist AI (RAG)

"This is the smartest feature. LexAssist uses a technique called **Retrieval-Augmented Generation**, or RAG. The idea is simple: instead of letting the AI invent answers, we force it to answer using the firm's own documents. When a lawyer uploads a document, the system sends it to a workflow that reads it, breaks it into pieces, and stores those pieces in a searchable knowledge base for that firm only. When the lawyer asks a question, the system finds the most relevant pieces and asks the AI to answer using them, and it shows the sources. So the answer is grounded in real documents — it is useful, not invented. And just like the rest of the app, each firm's knowledge base is separate."

Slide 12 — Deployment, CI/CD & Reliability

"Finally, deployment. The whole platform runs on a **completely free cloud stack** — no credit card needed — yet it is built like a real production system. The website is on Vercel, the API runs in a Docker container on Render, the database is on Neon, the cache and queues use Upstash Redis, documents are on Supabase storage, and the AI workflow is on n8n Cloud."

"Every time I change the code and push it, an automatic pipeline checks that it builds correctly, and then the new version deploys by itself. I also added a small scheduled task that 'wakes up' the free server every few minutes so it is always ready, and the app detects a weak internet connection and warns the user with a red Wi-Fi icon. This makes it reliable even on slow networks."

Slide 13 & 14 — Results / Demo

"Now I will show you the live, deployed application. I will register a firm, log in, create a case, upload a document, search for it, ask LexAssist a question, and send a notification — so you can see that everything I described actually works."

(Tip: keep the app already ‘warmed up’ in another tab before you start, so there is no waiting.)

Slide 15 — Conclusion

“To conclude: LexManage takes the real, everyday problems of a law firm — scattered information, lost documents, missed deadlines and slow research — and answers each one with a concrete feature. It is secure and private by design, its AI gives grounded answers from the firm’s own files, and it is fully deployed on a free, modern cloud stack. Thank you for listening. I am happy to answer your questions.”

Part 2 — The Technology Stack and Why Each Tool Was Chosen

For every tool, here is what it does and — more importantly — **why** you chose it. In a defense, the jury cares less about the name and more about your reason. Always answer with a reason.

Frontend (what the user sees)

Technology	What it does	Why it was chosen
React 19	Builds the user interface as small reusable pieces called components.	It is the most popular UI library, it is fast, and components keep the code clean and easy to maintain.
Vite	The build tool that compiles and serves the React app.	It is extremely fast and gives instant reload while coding, which made development much quicker.
Tailwind CSS	A styling system based on small utility classes.	It let me build a consistent, professional, responsive design quickly, including dark mode, without messy CSS files.
Zustand	Stores global state like the logged-in user and chosen language.	It is very small and simple compared to alternatives, and it keeps the user logged in across browser tabs.
React Query	Manages all data coming from the server: caching, refetching, retries.	It removes a lot of manual work, makes the app feel fast through caching, and handles slow networks with retries.
React Hook Form + Zod	Handles forms and checks that input is valid.	Zod lets me use the SAME validation rules on the frontend and backend — one single source of truth for correct data.
React Router	Moves between pages without reloading.	It gives a smooth single-page-app feel and lets me protect admin pages by role.
Socket.io client	Receives live updates from the server.	It powers instant notifications so users see new alerts without refreshing.

Backend (the brain of the system)

Technology	What it does	Why it was chosen
NestJS	The framework that structures the whole server into modules.	It enforces clean, professional organisation (one module per feature) and has built-in support for security, validation and websockets.
TypeScript	JavaScript with type-checking.	It catches many mistakes before the code even runs, which makes a big project safer and easier to trust.
PostgreSQL	The main relational database.	It is powerful, reliable, free and open-source, and perfect for the structured, related data of a law firm.

Prisma (ORM)	Translates my code into safe database queries.	It is type-safe and automatically protects against SQL injection, so the database layer is secure by default.
JWT	Tokens that prove who a user is after login.	They let the server stay stateless and scalable — no session storage needed — and they carry the firm ID for isolation.
bcrypt	Hashes (scrambles) passwords before storing them.	Even if the database leaked, passwords cannot be read. It is the industry standard for password safety.
BullMQ + Redis	Runs background jobs like scheduled reminders and emails.	It lets the app schedule a notification for a future date and send it automatically, without blocking the user.
Socket.io (server)	Pushes real-time events to connected users.	Needed for instant notifications; clients join a per-firm room so messages never leak across firms.
Helmet + Throttler	Adds security headers and limits request rate.	Helmet hardens the app against common web attacks; the throttler blocks abuse and brute-force attempts.

Data, storage, AI & infrastructure

Technology	What it does	Why it was chosen
Supabase Storage (S3)	Stores the actual document files.	It is S3-compatible, has a free tier, and lets me organise files per firm and serve them through secure expiring links.
Redis (Upstash)	In-memory cache and job queue backend.	It makes repeated requests fast and powers the scheduled-notification system, on a free managed plan.
n8n Cloud	The workflow that powers the AI (RAG).	It lets me build the document-ingestion and question-answering pipeline visually, and keeps AI logic separate from the app.
Resend	Sends transactional emails.	It reliably delivers urgent notifications by email with very little setup.
Docker	Packages the backend so it runs the same everywhere.	It removes ‘it works on my machine’ problems and makes cloud deployment simple and repeatable.
Neon (PostgreSQL)	The hosted, serverless database.	It gives a real PostgreSQL database for free, scales automatically, and needs no server maintenance.
Render	Hosts the backend API.	It deploys a Docker container for free and redeploys automatically when I push new code.
Vercel	Hosts the frontend website.	It serves the React app worldwide on a fast CDN, for free, with automatic deployment.

GitHub Actions	Runs the automatic test-and-deploy pipeline (CI/CD).	It checks every code change builds correctly before it goes live, so a broken version never reaches users.

Part 3 — Jury Questions and Answers

These are the questions a jury is likely to ask, with clear answers in simple English. Read each answer, then say it in your own words. If you do not know something, it is fine to say: “That is a good point — in the current version I handled X, and Y would be a strong next step.” Confidence and honesty win marks.

3.1 General & Conceptual Questions

Q. In one sentence, what is LexManage?

A. It is a secure, multi-tenant web platform that lets a law firm manage its cases, documents, clients, deadlines and notifications in one place, with an AI assistant that answers legal questions from the firm’s own documents.

Q. What does “multi-tenant” mean, and why does it matter here?

A. Multi-tenant means many separate organisations (law firms) use the same single running application, but each one’s data is completely isolated. It matters because law firm data is highly confidential — one firm must never see another’s clients, cases or files. It also means I host one app for everyone instead of installing separate software for each firm, which is far cheaper and easier to maintain.

Q. What does “SaaS” mean?

A. Software as a Service. The firm does not install or maintain anything; they just open a web browser and use the app. I run and update the software centrally in the cloud. This suits firms with little or no IT staff.

Q. Who are the users of the system?

A. The staff of a law firm, in five roles: Super Admin, Cabinet (Firm) Admin, Lawyer, Assistant and Secretary. Each role sees and can do only what its job requires.

Q. Why did you choose the legal domain specifically?

A. Because legal work is document-heavy, deadline-driven and highly confidential — exactly the kind of work that suffers most from paper files and scattered tools. It is a domain where good software has a clear, measurable impact, and where many firms, especially in Africa, are still under-served by affordable digital tools.

Q. Is this a real, working application or just a prototype?

A. It is a real, fully deployed application running live on the internet on a production-shaped cloud stack, with a database, file storage, real-time notifications and CI/CD. It is an MVP — a minimum viable product — meaning the core is complete and working.

Q. What makes your project different from a normal CRUD app?

A. Three things: true multi-tenant data isolation enforced on every request; an AI assistant grounded in each firm’s own documents using RAG; and a production-grade, secure, automatically-deployed cloud architecture. It is not just create-read-update-delete; it is a secure, real-time, AI-enabled SaaS.

3.2 Security Questions (expect many of these)

Q. How exactly do you keep one firm’s data separate from another’s?

Every user's login token (JWT) contains a **tenantId** — the ID of their firm. On every request, a middleware verifies the token and reads that tenantId. The application then attaches that tenantId to every database query, so the database only ever returns rows belonging to that firm. Files are stored under a folder named after the tenantId, and real-time messages go to a per-firm 'room'. So isolation is enforced at the data layer, the storage layer and the real-time layer — not just hidden in the interface.

Q. How does login and authentication work?

A. When a user logs in, the server checks the email and the bcrypt-hashed password. If correct, it issues two tokens: a short-lived access token (15 minutes) and a longer refresh token (7 days). The access token is sent with every request to prove identity. When it expires, the refresh token quietly gets a new one, so the user stays logged in without re-typing their password.

Q. Why two tokens instead of one? Isn't that more complex?

A. It is a deliberate security trade-off. The access token is short-lived, so if it is ever stolen it is only useful for a few minutes. The refresh token is stored more safely and can be revoked. This limits the damage of a stolen token while keeping the user logged in conveniently.

Q. How are passwords stored?

A. They are never stored as plain text. They are hashed with bcrypt using a cost factor of 12. Hashing is one-way, so even if the database were stolen, the real passwords cannot be recovered. Bcrypt is also deliberately slow, which makes brute-force guessing impractical.

Q. Could someone forge a token and pretend to be another firm?

A. No. Each token is cryptographically signed with a secret key only the server knows, and I pin the algorithm to HS256 so an attacker cannot trick the server into accepting an unsigned token. If even one character of the token is changed, the signature check fails and the request is rejected.

Q. How do you prevent SQL injection?

A. I never build SQL by joining strings. I use Prisma, an ORM that sends every value as a separate parameter, so user input can never change the meaning of a query. This makes standard SQL injection impossible by design.

Q. How do you protect against cross-site and common web attacks?

A. I use Helmet, which sets protective HTTP headers. I lock CORS so the API only accepts requests from my official website address. All traffic is forced over HTTPS. And every input is validated and 'whitelisted' — unexpected fields are rejected automatically.

Q. What stops someone from spamming or brute-forcing your API?

A. Rate limiting. I use a throttler that allows at most about 10 requests per second, 60 per minute, and 600 per hour per client. Beyond that, requests are blocked. This slows down brute-force password guessing and protects the server from abuse.

Q. How are documents protected? Can someone guess a file URL?

A. Files are private. They are not served by a public link. To download a file, the server generates a temporary **signed URL** that works for only 15 minutes and then stops working. You also need a valid login and the right role even to request that link. So guessing a URL is useless.

Q. What about role-based access — can a secretary delete a case?

A. No. Access is controlled by Role-Based Access Control. Sensitive actions are protected by guards that check the user's role at both the routing level and the API level. A secretary or assistant simply does not have permission for admin-only actions, and the server rejects the request even if they tried

to call the API directly.

Q. Where do you validate input — frontend or backend?

A. Both, but the backend is the real guard. The frontend validates for a nice user experience, but the backend re-validates everything with strict schemas, because a frontend check can always be bypassed. The server never trusts the client.

Q. Is the AI a security risk? Could it leak one firm's documents to another?

A. No. Each firm's documents are stored in a separate knowledge base keyed by tenantId. When a question is asked, the request carries the firm's ID, and only that firm's content is searched. So the AI cannot answer using another firm's documents.

Q. What happens if the server restarts — are users logged out?

A. No. Authentication is stateless: the proof of identity lives in the signed token held by the browser, not in server memory. So the server can restart or scale to many instances and users stay logged in.

Q. Do you log security-sensitive actions?

A. Yes. There is an audit log model that records sensitive actions with the user, the action and a timestamp, which gives traceability — important for a legal system where accountability matters.

Q. What are the main security weaknesses you are aware of?

A. Honestly, a few areas I would harden next: adding two-factor authentication for logins, encrypting specific sensitive fields at rest in the database, adding automated security scanning in the pipeline, and a formal penetration test. The foundations — isolation, hashing, signed tokens, RBAC, rate limiting — are already in place.

3.3 How LexManage Solves Problems in African Law Firms

Q. Why is this project especially relevant for African law firms?

Because many African firms face a specific combination of challenges: tight budgets, limited IT staff, unreliable internet and power, heavy reliance on paper, and — in countries like Cameroon — the need to work in **both French and English**. LexManage was designed around exactly these realities: it is free to run on the cloud, needs no local server or IT team, works on a phone, survives weak connections, and is fully bilingual.

Q. How does it address the low budgets of small African firms?

A. The entire platform runs on free cloud tiers, with no expensive licenses and no on-premise servers to buy or maintain. A small firm can use professional-grade legal software at almost no cost, which removes the biggest barrier to going digital.

Q. Many areas have unreliable internet. How does the app cope?

A. I built in network resilience. The app detects a weak or lost connection and shows a clear red Wi-Fi warning so the user knows it is the network, not the app. Requests automatically retry instead of failing instantly. And because the free server can 'sleep', I added a keep-alive ping and a friendly 'waking up the server' message with automatic retry, so the user is never stuck on a blank error.

Q. Why is the bilingual feature important here?

A. Several African countries are officially bilingual or mixed-language. Cameroon, for example, uses both French and English in its legal system. LexManage switches the entire interface — including document categories — between French and English instantly, so a firm can work in whichever language a given client or court requires. Most foreign legal software is English-only, which excludes

Francophone users.

Q. Many lawyers work from their phones. Does the app support that?

A. Yes. The interface is fully responsive and mobile-first. The header, search, notifications and chatbot were specifically adjusted to work well on small screens, so a lawyer can check a case or a deadline from a phone, not only a computer.

Q. African firms often have no IT department. Who maintains the system?

A. Nobody at the firm has to. Because it is SaaS on managed cloud services, all updates, backups, scaling and server maintenance happen centrally and automatically. The firm just logs in and works.

Q. How does it solve the paper-file problem common in African firms?

A. It digitises the whole case file: the case details, the linked client, the deadlines and all the documents live together online, searchable in seconds. No more lost binders, no more driving to the office to find one paper, and no more single physical copy that can be destroyed by fire or water.

Q. Confidentiality is a big concern. How does it build trust with firms?

A. Through strict isolation and security: each firm's data is walled off, documents are private and only reachable through short-lived signed links, passwords are hashed, tokens are signed, and roles limit who can see what. A firm can trust that its clients' secrets stay its own.

Q. Could this scale to many firms across a country or region?

A. Yes. That is the point of the multi-tenant SaaS design and the cursor-based pagination: adding more firms just means more tenants in the same system, and the architecture and database queries stay efficient as data grows. The cloud services scale automatically.

Q. Is there a real market for this?

A. Yes. There are thousands of small and medium law firms across Africa that cannot afford expensive foreign legal software and are still on paper. A free-to-run, bilingual, mobile-friendly, secure platform fills a clear gap.

3.4 Architecture & Technical Questions

Q. Why did you separate the frontend and backend instead of one combined app?

A. Separation of concerns. The frontend focuses only on the user experience, and the backend focuses on data, rules and security. This makes each side simpler to build and test, lets them scale independently, and means I could deploy the website and the API on the platforms each is best suited to.

Q. Why NestJS and not plain Node.js or Express?

A. NestJS gives structure. It organises code into modules with clear boundaries and has built-in, professional support for validation, authentication, guards, websockets and dependency injection. For a project this size, that structure keeps the code clean and maintainable, where plain Express would become messy.

Q. Walk me through what happens when a user opens their list of cases.

The browser sends a request to the API with the user's access token. A middleware verifies the token and reads the firm ID. A guard checks the user is authenticated. The cases controller calls the cases service. The service asks Prisma for cases belonging to that firm only, using cursor-based pagination. The result may be served from the Redis cache if it is fresh. The data is returned as JSON, and React Query on the frontend caches it and shows it. Every step is firm-scoped and permission-checked.

Q. How do real-time notifications work technically?

A. I use a Socket.io gateway. When a user connects, they join a room named after their firm and a room for themselves. When a notification is created, the server emits an event to the right room, and connected browsers receive it instantly and update the bell icon — no page refresh needed.

Q. What is the role of Redis in your system?

A. Two roles. First, it is a cache: frequently requested data is stored briefly so repeated requests are fast. Second, it is the backbone of the job queue (BullMQ), which lets me schedule a notification to be sent at a future time and send urgent emails in the background.

Q. What design patterns did you use?

A. Mainly the modular controller–service pattern that NestJS encourages: controllers handle the HTTP layer, services hold the business logic, and Prisma handles data. I also used dependency injection throughout, middleware for cross-cutting concerns like tenancy, and guards for authorisation.

3.5 Artificial Intelligence (RAG) Questions

Q. What is RAG, in simple terms?

A. RAG means Retrieval-Augmented Generation. Instead of letting the AI answer from its general memory, we first **retrieve** the most relevant pieces of the firm's own documents, and then ask the AI to **generate** an answer using only those pieces. So the answer is based on real documents, not guesswork.

Q. Why use RAG instead of just asking a chatbot like ChatGPT?

A. A general chatbot does not know your firm's private case files, and it can 'hallucinate' — confidently invent wrong answers. For legal work that is dangerous. RAG grounds every answer in the firm's actual documents and can show the sources, which is accurate, private and trustworthy.

Q. How does a document get into the AI's knowledge?

A. When a lawyer uploads a document, the backend sends it to an n8n workflow. That workflow reads the text, splits it into small chunks, converts each chunk into a numerical form (an embedding), and stores them in a knowledge base tagged with the firm's ID. Later, a question is matched against those chunks to find the most relevant ones.

Q. Why did you use n8n for the AI instead of coding it directly?

A. n8n let me build the ingestion and question-answering pipeline as a clear visual workflow, separate from the main app. This keeps the AI logic modular and easy to change, and it means the heavy AI processing does not block or complicate the main API.

Q. What if the AI gives a wrong answer?

A. Because answers are grounded in the firm's documents and show their sources, the lawyer can verify them. The interface also reminds users that the AI assists but does not replace professional judgement. RAG greatly reduces, though never fully eliminates, wrong answers — so the human stays in control.

3.6 Database, Data & Performance Questions

Q. Why PostgreSQL and not a NoSQL database like MongoDB?

A. Law firm data is highly relational — cases belong to firms, documents belong to cases, deadlines belong to cases, and so on. PostgreSQL is built for these relationships and guarantees consistency. A

NoSQL database would make these links harder to keep correct. For structured, related, important data, a relational database is the right choice.

Q. What is an ORM and why use Prisma?

A. An ORM (Object-Relational Mapper) lets me work with the database using clean code instead of raw SQL. Prisma is type-safe, so mistakes are caught early, and it parameterises every query, which prevents SQL injection. It also makes the data model easy to read and evolve.

Q. You used cursor-based pagination. What is it and why?

A. Old-style pagination uses OFFSET, which makes the database scan and skip all earlier rows — this gets slower as data grows. Cursor-based pagination instead remembers the last item seen and jumps straight to the next batch using the index. It stays fast no matter how many records exist, which matters for a system meant to grow with many firms.

Q. How do you keep the app fast?

A. Several ways: a Redis cache for repeated reads, cursor pagination for large lists, code-splitting so the browser only loads the page it needs, and database indexes on the fields used for filtering and tenancy. Together these keep the app responsive.

Q. How many tables are in your database and what are the main ones?

A. There are 13 models. The main ones are Tenant (the firm), User, Case, Client, Document, Deadline, Notification, plus supporting ones like ScheduledNotification, NotificationTemplate, ChatConversation, ChatMessage, Invitation and AuditLog.

Q. How do you handle database changes as the project evolves?

A. Through Prisma's schema and migration system. I describe the data model in one schema file, and Prisma generates the matching database structure. This keeps the code and the database in sync and version-controlled.

3.7 Deployment, DevOps & Reliability Questions

Q. What is CI/CD and what does yours do?

A. CI/CD means Continuous Integration and Continuous Deployment. Every time I push new code, GitHub Actions automatically checks that the backend type-checks and builds and that the frontend builds. If anything fails, it does not deploy — so a broken version never reaches users. If it passes, Vercel and Render deploy the new version automatically.

Q. Why did you containerise the backend with Docker?

A. Docker packages the backend with everything it needs, so it runs exactly the same on my machine and on the cloud server. This removes 'it works on my computer' problems and makes deployment predictable and repeatable.

Q. You used only free services. Doesn't that make it unreliable?

The free tiers have one real limitation: the backend 'sleeps' after 15 minutes of inactivity and takes a moment to wake. I handled this directly: a scheduled keep-alive ping keeps it awake, and if it ever is asleep, the app automatically retries and shows a friendly 'waking up' message instead of an error. The architecture itself is production-shaped — the same code can move to paid tiers for instant scaling with no redesign.

Q. What happens if the backend is slow or down when a user logs in?

A. The frontend does not just fail. It retries the request several times with increasing delays, shows a ‘waking up the server’ banner so the user understands, and detects whether the problem is actually the user’s own weak internet, in which case it shows a red Wi-Fi warning. The user is never left staring at a blank error.

Q. How would you scale this if 500 firms joined tomorrow?

A. The design is already multi-tenant and stateless, so I would move the database, Redis and backend to paid tiers that scale automatically, and because the backend is stateless and Dockerised, I can run several copies behind a load balancer. The cursor pagination and caching keep queries efficient as data grows. No redesign is needed — just bigger resources.

Q. How do you manage secrets like database passwords and API keys?

A. They are never written in the code or committed to GitHub. They are stored as environment variables in each cloud platform’s dashboard, and the code reads them at runtime. The repository only contains an example file with the variable names, not the real values.

3.8 Testing, Limitations & Future Work

Q. How did you test the application?

A. I tested it in three main ways: type-checking with TypeScript catches a whole class of errors before running; the CI pipeline verifies that both the backend and frontend build correctly on every change; and I did thorough manual end-to-end testing of every feature on the deployed app — registering, logging in, creating cases, uploading documents, searching, using the AI and sending notifications.

Q. What are the current limitations of your project?

A. It is an MVP, so some areas are deliberately basic: there is limited automated unit-test coverage; the free hosting tier sleeps; billing and time-tracking are not yet built; and security could go further with two-factor login and field-level encryption. I know these limits, which is itself part of good engineering.

Q. If you had three more months, what would you add?

A. Two-factor authentication, automated unit and integration tests in the pipeline, a billing and invoicing module, offline support so lawyers can work without internet and sync later, and a mobile app version. I would also add field-level encryption for the most sensitive data.

Q. What was the hardest part of building this?

A. Getting multi-tenant isolation right and trustworthy — making absolutely sure that every single query, file path and real-time message is scoped to the correct firm — and then deploying a full, secure, real-time stack on free services while handling the cold-start problem gracefully. Both required careful, layered thinking.

Q. What did you learn from this project?

A. I learned how to design and build a complete, secure, real-world system end-to-end: data modelling, API design, authentication and authorisation, real-time communication, AI integration, performance tuning, and cloud deployment with CI/CD. Most importantly, I learned to think about security and the real user’s context from the very start, not as an afterthought.

Q. Why should we be impressed by this project?

A. Because it is not a toy. It is a complete, live, secure, multi-tenant SaaS that solves a real and important problem, uses a modern professional stack correctly, integrates grounded AI, and is thoughtfully adapted to the real conditions of African law firms — all deployed for free with automatic

delivery. It shows I can take an idea from a database diagram to a running, useful product.

Part 4 — Suggested Additions & Final Defense Tips

Here are extra things you can add to your slides or notes to look even more prepared. You can also feed these into NotebookLM so it can quiz you on them.

Things worth adding to your presentation

- **A simple architecture diagram** on screen while you talk — even a hand-drawn boxes-and-arrows version helps the jury follow.
- **An Entity-Relationship (ER) diagram** of the 13 database tables — juries love seeing the data model. You already have one in the repo.
- **A short live demo plan** written on a card, in order, so you never freeze: register → login → create case → upload document → search → ask LexAssist → send notification.
- **One concrete number** about the problem (e.g. 'a missed filing deadline can dismiss an entire case') to make the problem feel real.
- **A 'security at every layer' slide** — list isolation, hashed passwords, signed tokens, RBAC, rate limiting, signed URLs. Security is your strongest selling point; show it proudly.
- **A 'why Africa' slide** — cost, bilingual (French/English), mobile-first, weak-network resilience, no IT staff needed. This connects your tech to real impact.
- **A limitations & future-work slide** — admitting limits with a clear plan makes you look mature, not weak.

Tips for the defense itself

- **Warm up the app before you start** so the cold-start delay never shows during the demo. Keep a tab open and logged in.
- **Always answer with a reason.** Not 'I used NestJS', but 'I used NestJS *because* it gives clean structure and built-in security'.
- **If you don't know something, stay calm.** Say what you did do, then describe how you would approach the gap. Never bluff.
- **Lead with security and impact.** When in doubt, steer the answer back to data isolation, confidentiality, and how it helps real African firms.
- **Use simple words.** The clearer you explain, the more it shows you truly understand it.
- **Have a backup of the demo** — a short screen recording or screenshots — in case the internet fails during your defense.
- **Know your numbers:** 17 backend modules, 13 database models, 5 roles, 2 languages, fully free deployment. These make you sound in command of your project.

One-line answers to keep ready

- **What is it?** A secure multi-tenant SaaS that runs a law firm's cases, documents, clients and reminders, with grounded AI.
- **What is special?** Real per-firm isolation, grounded RAG AI, and a full secure cloud deployment — built for African firms' real conditions.
- **Why secure?** Isolation per firm, hashed passwords, signed tokens, role permissions, rate limiting, and expiring document links.
- **Why for Africa?** Free to run, bilingual, mobile-first, and resilient on weak internet, with no IT staff required.

You built a complete, real, secure system. Speak about it with calm confidence — you know it better than anyone in the room.